

Auto-Encoding Variational Bayes: Further Explorations on Reparameterization Trick

Shenghan Gao, Mokai Pan, *Undergraduate, ShanghaiTech University*

Abstract—Deep generative models, particularly Variational Autoencoders (VAEs), have revolutionized machine learning by integrating probabilistic inference with neural networks, enabling data reconstruction and sample generation. This study explores the theoretical and practical aspects of the reparameterization trick—a pivotal innovation that addresses the challenge of differentiating through stochastic latent variables during VAE training. Extending its applications to generative models like diffusion models, we analyze its role in optimizing the variational lower bound and controlling noise during data generation. Experimental results on the MNIST dataset highlight the potential and limitations of VAEs in tasks like image reconstruction and generation. Despite promising outcomes, we observe areas for improvement in the network architecture and model performance, paving the way for future advancements in generative modeling.

Index Terms—Neural Networks, Generative Models,

I. INTRODUCTION

Deep generative models have become a cornerstone in the field of machine learning, providing powerful tools for learning complex, high-dimensional data distributions. Among these models, Variational Autoencoders (VAEs) stand out due to their principled combination of probabilistic inference and neural networks. VAEs enable both the reconstruction of observed data and the generation of new samples, making them widely applicable in domains such as image generation, data augmentation, and unsupervised learning.

A critical component of VAE training is the reparameterization trick, which addresses the challenge of backpropagating through stochastic latent variables. This technique transforms the sampling process into a deterministic computation, allowing for efficient optimization of the variational lower bound. Despite its foundational role in VAEs, the broader applications and theoretical implications of the reparameterization trick remain active areas of exploration, particularly in the context of other generative models such as diffusion models.

This work aims to deepen the understanding of the reparameterization trick and its applications in generative modeling. By revisiting its theoretical foundations and investigating its extensions, we seek to provide insights that can inform both the development of new models and the refinement of existing ones.

II. AUTO-ENCODING VARIATIONAL BAYES

A. Data Setup and Model Assumption

Let the dataset be denoted by $\mathbf{X} = \{\mathbf{x}^{(i)}\}_{i=1}^N$, where N represents the number of independent and identically distributed (i.i.d.) samples of a continuous or discrete variable $\mathbf{x}$. Assume that the data is generated through a random process involving an unobserved continuous latent variable $\mathbf{z}$. The data generation process consists of two key steps:

- 1) A value $\mathbf{z}^{(i)}$ is generated from a prior distribution $p_{\theta^*}(\mathbf{z})$
- 2) A value $\mathbf{x}^{(i)}$ is generated from the conditional distribution $p_{\theta^*}(\mathbf{x}|\mathbf{z})$

where the prior $p_{\theta^*}(\mathbf{z})$ and likelihood $p_{\theta^*}(\mathbf{x}|\mathbf{z})$ come from parametric families of distributions $p_{\theta}(\mathbf{z})$ and $p_{\theta}(\mathbf{x}|\mathbf{z})$ and that their PDFs are differentiable almost everywhere w.r.t. both θ and $\mathbf{z}$.

B. The Variational Bound

The marginal likelihood is composed of a sum over the marginal likelihoods of individual datapoints $\log p_{\theta}(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}) = \sum_{i=1}^N \log p_{\theta}(\mathbf{x}^{(i)})$, through some tricks,

$$\begin{aligned} \log p_{\theta}(\mathbf{x}^{(i)}) &= \int q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) \log p_{\theta}(\mathbf{x}^{(i)}) d\mathbf{z} \\ &= \int q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})} d\mathbf{z} \\ &= D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) || p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})) + \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \end{aligned} \quad (1)$$

where the first RHS term is the KL divergence of the approximate from the true posterior, the second RHS term is called the (variational) lower bound on the marginal likelihood of datapoint i :

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} \left[-\log q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) + \log p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z}) \right] \\ &= \int q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} d\mathbf{z} \\ &= \int q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) \log \frac{p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}) p_{\theta}(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} d\mathbf{z} \\ &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} \left[\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}) \right] \\ &\quad - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) || p_{\theta}(\mathbf{z})) \end{aligned} \quad (2)$$

In the courses we have learned that for any two probability distributions p and q , the KL divergence between the two probability distributions is always positive:

$$D_{KL}(p||q) \geq 0 \quad (3)$$

Hence,

$$\log p_{\theta}(\mathbf{x}^{(i)}) \geq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \quad (4)$$

which the reason why $\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$ is called variational lower bound.

Our target is to differentiate and optimize the lower bound $\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$ w.r.t. both the variational parameters ϕ and generative parameters θ , but unluckily, to get $\frac{\partial \mathcal{L}}{\partial \phi}$ is hard. As a result, the following algorithm is designed.

C. SGVB estimator and AEVB algorithm

This section proposes a generalized solution algorithm for AEVB (Auto-Encoding Variational Bayes), which attempts to infer and learn the hidden distribution of $\mathbf{z}$ and to achieve data generation by sampling $\mathbf{z}$.

For a chosen approximate posterior $q_{\phi}(\mathbf{z}|\mathbf{x})$, we can reparameterize the random variable $\tilde{\mathbf{z}} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$ using a differentiable transformation $g_{\phi}(\cdot, \mathbf{x})$ of an (auxiliary) noise variable: $\tilde{\mathbf{z}} = g_{\phi}(\epsilon, \mathbf{x})$ with $\epsilon \sim p(\epsilon)$, which is explained in the following section II-D.

We can now form Monte Carlo estimates of expectations of some function $f(\mathbf{z})$ w.r.t. $q_{\phi}(\mathbf{z}|\mathbf{x})$ as follows:

$$\begin{aligned} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} [f(\mathbf{z})] &= \mathbb{E}_{p(\epsilon)} [f(g_{\phi}(\epsilon, \mathbf{x}^{(i)}))] \\ &\simeq \frac{1}{L} \sum_{l=1}^L f(g_{\phi}(\epsilon^{(l)}, \mathbf{x}^{(i)})) \end{aligned} \quad (5)$$

where $\epsilon^{(l)} \sim p(\epsilon)$.

Then, the Stochastic Gradient Variational Bayes (SGVB) estimator $\tilde{\mathcal{L}}^A(\theta, \phi; \mathbf{x}^{(i)}) \simeq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$ comes:

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} [-\log q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) + \log p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})] \\ &\simeq \frac{1}{L} \sum_{l=1}^L [\log p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z}^{(i,l)}) - \log q_{\phi}(\mathbf{z}^{(i,l)}|\mathbf{x}^{(i)})] \\ &\triangleq \tilde{\mathcal{L}}^A(\theta, \phi; \mathbf{x}^{(i)}) \end{aligned} \quad (6)$$

In addition, the author claimed that often the KL-divergence $D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z}))$ can be integrated analytically, so in Eqn. (2) we only need to estimate the term $\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})]$ which yields the second version of the SGVB estimator $\tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) \simeq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$:

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})} [\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})] \\ &\quad - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z})) \\ &\simeq \frac{1}{L} \sum_{l=1}^L (\log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)})) \\ &\quad - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\theta}(\mathbf{z})) \\ &\triangleq \tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) \end{aligned} \quad (7)$$

where $\mathbf{z}^{(i,l)} = g_{\phi}(\epsilon^{(i,l)}, \mathbf{x}^{(i)})$ and $\epsilon^{(l)} \sim p(\epsilon)$.

Given multiple datapoints from a dataset $\mathbf{X}$ with N datapoints, we can construct an estimator of the marginal likelihood lower bound of the full dataset, based on minibatches:

$$\mathcal{L}(\theta, \phi; \mathbf{X}) \simeq \tilde{\mathcal{L}}^M(\theta, \phi; \mathbf{X}^M) = \frac{N}{M} \sum_{i=1}^M \tilde{\mathcal{L}}(\theta, \phi; \mathbf{x}^{(i)}) \quad (8)$$

Derivatives $\nabla_{\theta, \phi} \tilde{\mathcal{L}}(\theta; \mathbf{X}^M)$ can be taken, and the resulting gradients can be used in conjunction with stochastic optimization methods such as Stochastic Gradient Descent (SGD) or Adaptive Gradient (Adagrad) [1].

D. Reparameterization Tricks

The reparameterization trick is a key technique in the AEVB algorithm, which addresses the challenge of optimizing the variational lower bound with respect to the parameters of the approximate posterior distribution. Traditionally, the gradients of the lower bound with respect to the variational parameters are difficult to compute due to the expectation under the approximate posterior, which often involves intractable integrals.

The reparameterization trick simplifies this problem by introducing a differentiable transformation that allows for the random variable sampled from the approximate posterior to be expressed as a deterministic function of an auxiliary noise variable. Specifically, if z is a continuous random variable with an approximate posterior distribution $q_{\phi}(z|x)$, the reparameterization trick enables us to write $z = g_{\phi}(\epsilon, x)$, where ϵ is an auxiliary noise variable with a prior known distribution $p(\epsilon)$, and g_{ϕ} is a vector-valued function parameterized by ϕ .

This reparameterization is crucial because it transforms the expectation under the intractable posterior into an expectation under a tractable distribution, which can be approximated using Monte Carlo methods, for instance,

$$\int q_{\phi}(\mathbf{z}|\mathbf{x}) f(\mathbf{z}) d\mathbf{z} \simeq \frac{1}{L} \sum_{l=1}^L f(g_{\phi}(\mathbf{x}, \epsilon^{(l)})) \quad \text{where } \epsilon^{(i)} \sim p(\epsilon) \quad (9)$$

By doing so, the gradients of the variational lower bound with respect to the parameters ϕ become computable, allowing for the use of standard stochastic gradient methods to optimize the lower bound.

E. An example: Variational Auto-Encoder (VAE)

In the example of VAE, we let the variational approximate posterior be a multivariate Gaussian with a diagonal covariance structure:

$$q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I}) \quad (10)$$

where the mean $\boldsymbol{\mu}^{(i)}$ and the covariance $\boldsymbol{\sigma}^{2(i)}$ are outputs of the encoding MLP.

Then we can sample from the posterior $\mathbf{z}^{(i,l)} \sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})$ by using reparameterization tricks $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)}$ where $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(0, \mathbf{I})$.

Meanwhile, the second version of the SGVB estimator can be formed as:

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) &\simeq \frac{1}{2} \sum_{j=1}^J \left(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2 \right) \\ &\quad + \frac{1}{L} \sum_{l=1}^L \log p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)}) \end{aligned}$$

where $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)}$ and $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(0, \mathbf{I})$ (11)

III. DIFFUSION MODELS AND ITS REPARAMETERIZATION TRICKS

Diffusion models, such as Denoising Diffusion Probabilistic Models (DDPM) [2], are another generative models that simulate the distribution of the data by adding noise step by step, and then generating the data by removing the noise step by step in an inverse process. The noise added at each step in the forward process needs to be precisely controlled to ensure that it returns to a reasonable number of data points.

Also, reparameterization tricks are involved in diffusion models during the process of noise introduction [3]. Often, a simple forward diffusion process which is a Markov chain can be expressed as,

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}) \quad (12)$$

If we would like to calculate the condition transition probability distribution $q(x_t|x_0)$, through the methods of Bayes formula and the property of Markov chains, the calculations might be

$$\begin{aligned} q(x_t|x_0) &= \int q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)dx_{t-1} \\ &= \int q(x_t|x_{t-1})q(x_{t-1}|x_0)dx_{t-1} \end{aligned} \quad (13)$$

Since we could induce that $q(x_t|x_0)$ is also Gaussian, then we may take the Mathematic Induction Methods to calculate $q(x_t|x_0)$, which is a bit difficult.

If we apply reparameterization tricks, the calculations might be much clearer and simple:

$$\begin{aligned} q(x_t|x_{t-1}) &= \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}) \\ \Rightarrow x_t &= \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t \quad \text{where } \epsilon_t \sim \mathcal{N}(0, \mathbf{I}) \end{aligned}$$

Denote $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, then

$$\begin{aligned} x_t &= \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t \\ &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t \\ &= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-1}) + \sqrt{1 - \alpha_t}\epsilon_t \\ &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t}\epsilon_{t-2} \\ &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\epsilon_{t-2} \\ &= \dots \\ &= \sqrt{\alpha_t\alpha_{t-1} \dots \alpha_1}x_0 + \sqrt{1 - \alpha_t\alpha_{t-1} \dots \alpha_1}\epsilon \\ &= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \end{aligned} \quad (14)$$

With the fact that the addition of the two Gaussians are also Gaussian: $\mathcal{N}(\mu_1, \sigma_1^2 \mathbf{I}) + \mathcal{N}(\mu_2, \sigma_2^2 \mathbf{I}) = \mathcal{N}(\mu_1 + \mu_2, (\sigma_1^2 + \sigma_2^2) \mathbf{I})$.

Hence, by applying the reparameterization tricks to the diffusion forward process, we easily obtained:

$$q(x_t|x_0) = \mathcal{N}(x_0; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (15)$$

and

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t) \quad (16)$$

Here the reparameterization tricks are used for controlling the noise introduction and removal process (forward and reverse process) to ensure the reversibility of the data generation process.

Eqn. (16) can be used for training the neural network ϵ_θ to predict the noise added in the steps of the forward process in the reverse process. Simply, after applying some Bayes formulas, we could get

$$\begin{aligned} q(x_{t-1}|x_t, x_0) &= \mathcal{N}(\mu_t(x_t, x_0), \Sigma_t^2 \mathbf{I}) \\ \mu_t(x_t, x_0) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 \\ \Sigma_t^2 &= \frac{(1 - \bar{\alpha}_{t-1})(1 - \alpha_t)}{1 - \bar{\alpha}_t} \end{aligned} \quad (17)$$

Take Eqn. (16) in,

$$\begin{aligned} \mu_t(x_t, x_0) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 \\ &= \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \frac{1 - \alpha_t}{1 - \bar{\alpha}_t}\epsilon_t) \end{aligned} \quad (18)$$

Hence, to predict the noise is equivalent to predict the mean value of $q(x_{t-1}|x_t, x_0)$, so we can generate the data from a prior distribution step by step. Here, to predict the mean $\mu_t(x_t, t)$ is equivalent to predict the noise ϵ_t , then, a part of the final loss looks like

$$\mathcal{L} = \mathbb{E}_{x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2]$$

In other words, the reparameterization tricks of DDPMs took the network parameter θ into the noise ϵ_θ at last, which is different from the tricks used in AEVB.

Overall, the key point of reparameterization tricks, both in VAE and DDPMs, is that it converts an otherwise stochastic sampling process into a deterministic function about the model parameters. Hence, the gradient can be propagated through this deterministic function, thus allowing the model parameters to be optimized according to the gradient of the loss function. This is a key technique for realizing gradient-based learning methods (e.g., back-propagation) in models involving stochastic sampling. The difference is that DDPMs took the network parameter θ into the noise ϵ_θ at last, while AEVB took the network parameters in the mean and the variance.

IV. EXPERIMENTS

We then reproduced the Variational Auto-Encoder model and conducted the experiments on MNIST dataset [4] constrained by equipment and time.

A. VAE

In VAE, we utilized a classic Variational Autoencoder (VAE) network architecture in the figure 1, which consists of an encoder and a decoder for learning the latent representation of the input data. Specifically:

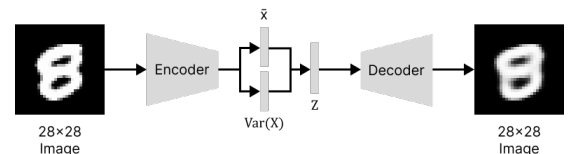


Fig. 1. Schematic diagram of network structure.

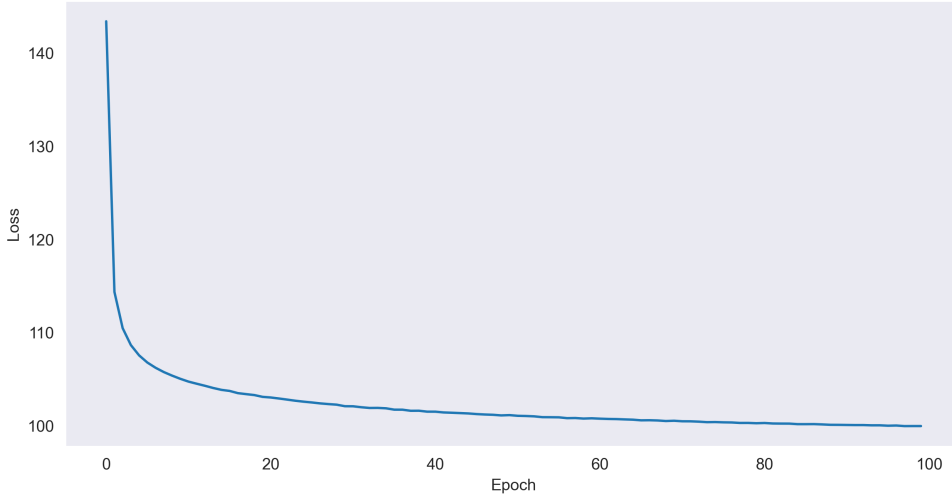


Fig. 2. Image Reconstruction Experiments on MNIST dataset: the curve of loss with epoch. This plot shows the decrease in training loss for the VAE model over 100 epochs during the image reconstruction task. The loss rapidly decreases initially, and then gradually levels off, suggesting convergence.

Encoder Our encoder part passes the input through a linear layer followed by a ReLU activation function, resulting in a 500-dimensional feature representation. This is then processed by two separate linear layers to produce the mean and log variance of the 20-dimensional latent space, which are crucial for the reparameterization trick.

Decoder Our decoder component starts with a ReLU-activated linear layer to expand the dimensionality, followed by a final linear layer with a Sigmoid activation to ensure the output values are valid pixel intensities, reconstructing the input image in a 28×28 pixel format.

Activation Functions ReLU activation functions are used in the intermediate layers of both the encoder and decoder to introduce non-linearity. A Sigmoid activation function is applied in the output layer of the decoder to ensure that the output values are between 0 and 1, suitable for representing pixel values.

We make attempts to apply our VAE models on two tasks: image reconstruction tasks and image generation tasks.

1) *VAE on Image Reconstruction:* The goal of image reconstruction is to learn a compressed representation of images and then generate reconstructed versions of the input images from this representation and to produce an output image that closely resembles the original, undamaged version, which is essential for enhancing visual data and enabling accurate analysis and interpretation.

We utilized the images in MNIST dataset consisting of gray-scale images of size 28×28 pixels, which were normalized to have pixel values between 0 and 1. Each image was flattened into a 784-dimensional vector before being fed into the VAE model.

After we trained a VAE model, we sampled an Ground-Truth origin image vector from the dataloader, input it into the trained VAE model and get the output reconstructed image.

We present the training progress in terms of epoch and loss in Figure 2, which illustrates the convergence of the model as the number of epochs increases. As can be seen, the total loss

decreases steadily, indicating effective learning of the model parameters.

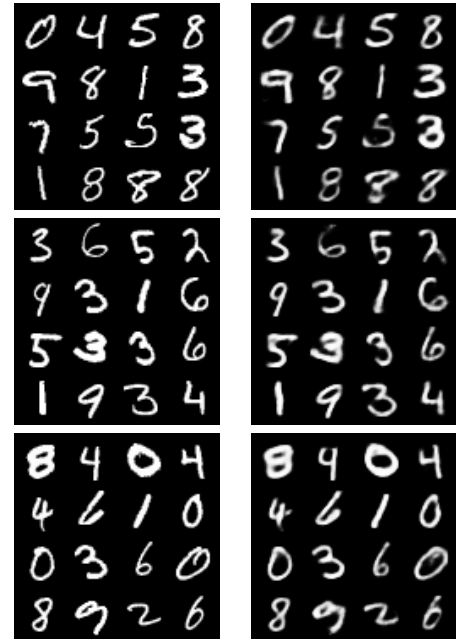


Fig. 3. Image Reconstruction Experiments on MNIST dataset: this figure presents a visual comparison between the original MNIST digits (left column) and their corresponding reconstructions by the VAE model (right column) with training different epochs, from top to bottom are 5, 15 and 50 epochs.

Figure 3 displays a set of pairs of the original images with their reconstructed images at specific epochs. After 50 epochs training, the VAE can basically reconstruct the original image successfully and the reconstruction results meets our expectations.

2) *VAE on Image Generation:* The image generation task involves the generation of new images by a computer program, which can be based on an existing dataset or created entirely by an algorithm. The VAE models are a type of generative model that can be effectively applied to the task of image

generation. The goal of the task is to generate images with high fidelity and high resolution that match the actual scene.

Different from the reconstructed task, we sampled a random noise vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ after training a VAE model, input it into the trained VAE model and get the output generated image.

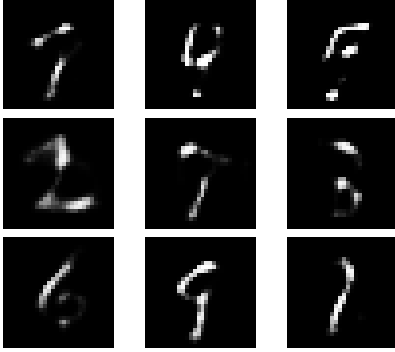


Fig. 4. Image Generation Experiments on MNIST dataset: this figure presents some examples of the generated images by the VAE model, obviously that the VAE-generated images are vague and we can only observe what an approximate number looks like.

Figure 4 displays some examples of the generated images by the VAE model with 50 epochs training. We found that the results of the generation seem not ideal. Some of our simple thoughts on this circumstances are that the neural networks we created are so simple as to a 784-dimensional data that it could not learn the whole information in the data distribution. What's more, insufficient latent spatial dimension (20-dimension) might lead to insufficient expressive ability.

V. CONCLUSION

Variational Auto-Encoder is a kind of generative model that combines a probabilistic graphical model with a deep neural network. Unlike traditional autocoders, VAE not only focuses on the reconstruction of the data, but also works on learning the underlying distribution of the data so that it can generate realistic new samples. The core idea of VAE is to map high-dimensional data to a low-dimensional latent space and to model probabilities in that space.

In the preceding sections, according to the several main methods II in Auto-Encoding Variational Bayes, we have reproduced some theories for VAE, conducted experiments on the MNIST dataset and analyzed some results and its related reasons.

As to the most important part of Auto-Encoding Variational Bayes, the reparameterization tricks, we discussed some differences between it acted upon Auto-Encoding Variational Bayes and Denoising Diffusion Probabilistic Models. The reparameterization tricks can be not only used to solve the problem that the gradient cannot be passed efficiently through random nodes, but also better controlled for the noise introduction to ensure the reversibility of the data generation process.

Also, there are still some shortcomings and much still to do. Our VAE models didn't perform well on image generation tasks, there's still much space for us to design a better-suited

neural networks for VAE models on the image generation tasks. We are looking forward to making further improvement of VAEs.

REFERENCES

- [1] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, no. 61, pp. 2121–2159, 2011. [Online]. Available: <http://jmlr.org/papers/v12/duchi11a.html>
- [2] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020. [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [3] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," 2016. [Online]. Available: <https://arxiv.org/abs/1602.07868>
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.